

Notes (Week 10 Monday)

These are the notes I wrote during the lecture.

This lecture is too long. Anything I don't get to, won't come on the exam.

Not every program that has a valid total correctness triple will **always** terminate. It **will** terminate if the precondition holds, but not necessarily if it's false.

Here's a valid total correctness triple:

$$\vdash [\perp] \text{ while true do skip od } [\top]$$

$$\{\phi\} P \{\psi\} \quad \{\phi\} Q \{\psi\}$$

$$\{\phi\} P + Q \{\psi\}$$

Formulas ϕ as part of $\mathcal{L}^+$ programs

- In partial correctness,
 - Perform a miracle that causes ϕ to be true
- In total correctness,
 - Abort unless ϕ is true

$$\{\phi\} \psi \{\phi \wedge \psi\}$$

...after we execute ψ , ψ is (somehow) true!

What then does $\perp$ mean?

- In partial correctness,
 - Make everything true!
- In total correctness world, abort

$P + \perp$	Either execute P, or make everything true
$\perp; P$	Make everything true, then execute P
$P + \top$	Either execute P, or make true true (aka do nothing)
$\top; P$	Make true true, then execute P

Give function calls meaning by using substitution

```
int f(int n) {
  if n = 0 then 1 else 0
}
```

Performing the function call

$f(0)$

The first step is to enter the body of the function
 Replace the function call with the body, but with
 all occurrences of the parameter(s) replaced
 by the argument(s)

Parameters: placeholders in the definition

Arguments: what you call the function with

$f(0)$

→

$(\text{if } n = 0 \text{ then } 1 \text{ else } 0)[n:=0]$

In the λ -calculus, you can write functions.

You can call functions,

but there's no data (other than functions)

(higher-order functions)

A computational model is Turing-complete

if it can represent every computable function.

What is a computable function?

Intuitively, a mathematical function (from $\mathbb{N}$ s $\mathbb{N}$ s)

such that an algorithm that is guaranteed to terminate
 can implement it

$(\lambda x. e)$ function abstraction
 a function with no name (anonymous function)
 with parameter x
 and body e

Function application

$f \ a$ function f applied to argument a

Reduction (computation),

because there's only functions,

there's only one computation step you can do:

call functions

$(\lambda x. x + 5) \ 3 \rightarrow (x + 5)[x:=3] = 3 + 5$

Unless we get to like slide 120,
the difference between variables and constants
doesn't really matter

$\lambda y. x. x y$

is sugar for

$\lambda y. \lambda x. x y$

$x y$	call x with argument y
$\lambda x. x y$	accepts an argument x , then calls that x with argument y
$\lambda y. \lambda x. x y$	accepts an argument y , and returns a function that accepts an argument x , then calls x with argument y

a function that accepts two arguments,
and calls the second with the first

Convention "currying" (named for Haskell Curry)

We don't have multi-argument functions.

But we don't need multi-argument functions.

Idea: encode $f(x,y) = x^y$
as $f = \lambda x. \lambda y. x^y$

We split the tuple and accept each component separately

The idea:

$(\lambda x y. x^y) 3 4$

I want this to reduce to

3^4

The way to get that is

$((\lambda x y. x^y) 3) 4$
 $\rightarrow (\lambda y. 3^y) 4$
 $\rightarrow 3^4$

Notice that

$(\lambda x y. x^y) (3 4)$

Is the wrong thing: 3 applied to 4 is probably not what we want

We only have functions,

but we want to encode data somehow to show that we can do useful computation.

In λ -calculus, it be what it do

Intuition: encode data in terms of its function,
 encode data as the thing we can do with it

What is a number? Ask Giuseppe Peano $Z, S Z, S(S Z)$

Peano gives a way to **construct** the natural numbers

..but here, we'll focus on how to **destruct** the natural numbers
 in other words: how to use them

When you have a number n , what do you with it?

idea: we use n to iterate some task n times (for-loop)

A Church numeral accepts two arguments:

$\lambda f x. \dots$

The first argument f represents the task to be repeated.

The second argument represents 0 , represents the initial object
 that the task should be performed on.

$0 = \lambda f x. x$

$1 = \lambda f x. f x$

$2 = \lambda f x. f(f x)$

$3 = \dots$

...so we can have numbers, but can we do stuff with the numbers?

The successor function $f(x) = x+1$ S

$\lambda n. \dots$ takes a number as input

$\lambda f x. \dots$ needs to return a number, which is a binary function
 $f(n f x)$

Addition $(\lambda x y. x + y)$

$(+) = \lambda n m. \lambda f x. n f (m f x)$

Let's do $1+1$. $(+) 1 1$

$(+) 1 1$

$= (\lambda n m. \lambda f x. n f (m f x)) 1 1$

$\rightarrow (\lambda m. \lambda f x. 1 f (m f x)) 1$

$\rightarrow \lambda f x. 1 f (1 f x)$

$= \lambda f x. 1 f ((\lambda f x. f x) f x)$

$\rightarrow \lambda f x. 1 f ((\lambda x. f x) x)$

$\rightarrow \lambda f x. 1 f (f x)$

$= \lambda f x. (\lambda f x. f x) f (f x)$

$\rightarrow \lambda f x. (\lambda x. f x) (f x)$

```
→ λf x. f (f x)
= 2
```

My goal here is not to convince you that you'd
 want to program in the λ -calculus
 The goal is to convince you that, in principle, you can

What do you *do* with booleans?

A: you choose a branch based on whether it's true or false
 (if, while)

We encode booleans as binary functions that return
 either the first or the second argument

```
true =  λx y. x
false = λx y. y
```

A boolean is essentially the same thing as an if-then-else

Whenever you see

```
b x y
```

Think:

```
if b then x else y
```

```
¬ = λb. b false true
```

```
∧ = λp q. p q false
```

```
p q false    read    "if p then q else false"
```

We can have booleans and nats. Can we do loops?

Or more clearly: unbounded repetition

Normal form.

A λ -calculus term is in *normal form* if it contains no β -redex.

A β -redex is a β -reducible subterm.

β -reduction is the fancy name for "apply a function call"

```
(λx. t) t' →β t[x:=t']
```

A λ term is in normal form if there are no more unevaluated
 function calls left.

The (vanilla) λ -calculus allows you to perform β -reductions
 anywhere in a term, in any order.

$$(\lambda x. (\lambda x. t) x) ((\lambda x. y) z)$$

This term has three β -redexes.

So we have three choices of where to reduce:

- reduce the argument before the top-level function call
 - eager evaluation, call-by-value
- perform the function call **before** evaluating the argument
 - call-by-name (e.g. Haskell)
- start evaluating inside the function before we call the function
 - compiler optimisations

Theorem:

if a lambda-term has a normal form, it's unique.

To rephrase "can we have loops" more formally, we might ask:

Does **every** λ -term have a normal form?

$f(x) = f(x-1) + 1$ $\leftarrow$ we can't do that,
because functions don't have names

But we **can** encode it.

First: a useless example of a term with no normal form.

$$(\lambda x. x x) (\lambda x. x x) \mapsto_{\beta} (x x)[x := \lambda x. x x] =$$

$$(\lambda x. x x) (\lambda x. x x)$$

That shows us that we can have unproductive loops. But what about loops (or at least recursive functions) that actually do stuff?

It just so happens that λ -calculus has a number of

fixpoint combinators

Y-combinator

Turing's fixpoint combinator T

$$Y f \mapsto_{\beta} f(Y f) \mapsto_{\beta} f(f(Y f)) \mapsto_{\beta} f(f(f(Y f)))$$

Imagine f is something like

"if n is 0 then return otherwise keep going"

$$Y = \lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))$$

$$(\lambda f. (\lambda x. f(x x)) (\lambda x. f(x x))) g$$

$$\mapsto_{\beta} (\lambda x. g(x x)) (\lambda x. g(x x))$$

$$\mapsto_{\beta} g((\lambda x. g(x\ x)) (\lambda x. g(x\ x)))$$

$$\equiv g(Y\ g)$$

Fix points are nice, because they allow us to encode loops
 But they're also the reason why λ -calculus fails as a foundation of logic

$$Y\ \neg = \neg(Y\ \neg)$$

In other words, there exists a term that equals its own negation

The solution (that we didn't do, but see COMP3151, COMP4161) is to add a type system to the λ -calculus

One last convention: the choice of placeholders doesn't matter.

$$(\lambda x. x) = (\lambda y. y)$$

Intuition: functions are equal (formally, α -equivalent) if they differ only in choice of placeholders for λ -bound arguments

This means that you're allowed to (and should) rename stuff to avoid variable capture before β -reducing

$$(\lambda y\ x. x+y)\ (3+x) =_{\alpha} (\lambda y\ z. z+y)\ (3+x)$$

$$\mapsto_{\beta} \lambda z. z + (3+x)$$

Q: Is α -equivalence computable?

A: Yes. by a (mostly straightforward) syntax walk